

DARTS: Differentiable Architecture Search

Hanxiao Liu¹ Karen Simonyan² Yiming Yang¹

¹*Carnegie Mellon University* ²*DeepMind*

Published at ICLR 2019

CS.50700 Paper review
TEAM 9

1. Motivation

Neural architecture search (NAS) has produced models that match or exceed hand-designed networks on image classification (Zoph et al., 2018; Real et al., 2018) and other tasks.

The bottleneck is computational cost.

- NASNet (Zoph et al., 2018) — reinforcement learning, $\approx 2,000$ GPU-days.
- AmoebaNet (Real et al., 2018) — regularized evolution, $\approx 3,150$ GPU-days.
- These methods treat architecture search as a black-box optimization problem over a discrete search space, requiring thousands of full architecture trainings.

Question: can architecture search be reformulated so that gradient-based optimization applies, reducing cost by orders of magnitude without sacrificing accuracy?

2. Related work and the bottleneck

Existing NAS methods fall into two broad families, both of which inherit the cost of evaluating each candidate architecture.

Black-box search over discrete architectures

- Reinforcement learning (Zoph & Le, 2017; Zoph et al., 2018)
- Evolution (Real et al., 2018; Liu et al., 2018b)
- Monte Carlo tree search (Negrinho & Gordon, 2017)
- Bayesian optimization (Kandasamy et al., 2018)

Cost-reduction strategies

- Restricted search spaces (Liu et al., 2018a)
- Performance prediction (Baker et al., 2018)
- Weight sharing (Pham et al., 2018b; Bender et al., 2018)
- Hypernetworks (Brock et al., 2018)

Common to all: the architecture is a discrete object, so search is non-differentiable. The proposed work attacks this assumption directly.

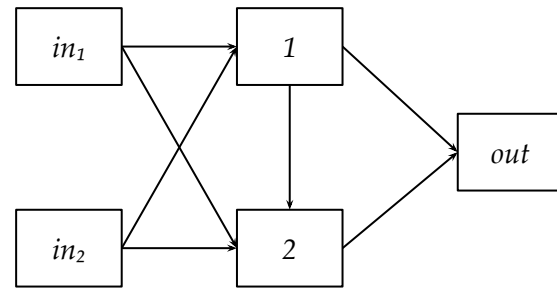
3. Search space: cells as directed acyclic graphs

What is the “architecture” in DARTS?

- DARTS searches for a **cell**, a small building block of the final network.
- A cell is represented as a directed acyclic graph (DAG).
 - Node: latent representation, e.g., feature map
 - Edge: candidate operation applied to a previous node
- Each edge chooses one operation from a candidate set:
 - convolution, pooling, skip connection, or zero.
- Thus, architecture search becomes choosing operations on the cell edges.

Key issue:

- This operation choice is **discrete and non-differentiable**.



Schematic of a cell with two inputs and two intermediate nodes.

4. Key idea: from discrete edges to continuous mixtures

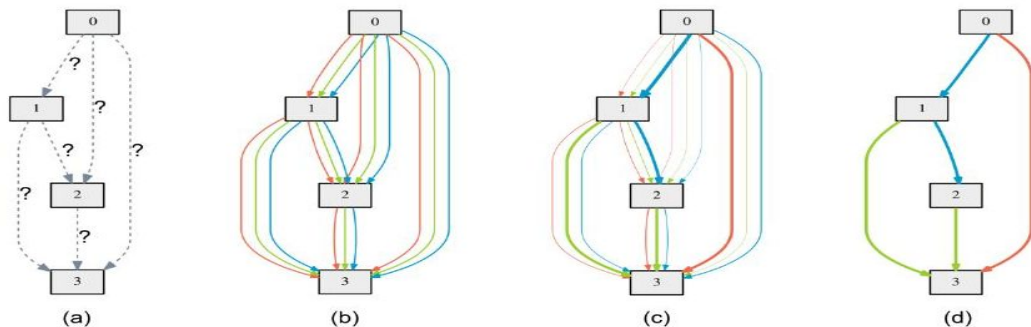


Figure 1 of Liu et al. (2019) — (a) operations on edges initially unknown; (b) continuous relaxation places a mixture of all candidate operations on each edge; (c) joint optimization of mixing weights and network weights; (d) discrete architecture induced by argmax.

The categorical choice of operation on each edge is replaced by a **continuous mixture**, parameterized by real-valued weights. The resulting objective is **differentiable in both the architecture parameters and the network weights**, so both can be trained by gradient descent.

5. Continuous relaxation

Let $\mathcal{O}$ denote the set of candidate operations.

For each edge (i, j) , DARTS replaces the **discrete operation choice** with a **softmax-weighted mixture**:

$$\bar{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x) \quad (2)$$

- α is the architecture parameter: one logit for each edge-operation pair.
- The mixed operation is differentiable with respect to α .
- Therefore, architecture search can be optimized by gradient descent.
- After search, the soft cell is converted into a discrete cell by selecting the strongest operations.

6. Bilevel optimization

After continuous relaxation, DARTS needs to optimize two types of parameters:

w : network weights

α : architecture parameters

Both L_{train} and L_{val} depend on w and α .

DARTS formulates architecture search as a **bilevel optimization problem**:

$$\min_{\alpha} \quad \mathcal{L}_{val}(w^*(\alpha), \alpha) \tag{3}$$

$$\text{s.t.} \quad w^*(\alpha) = \operatorname{argmin}_w \mathcal{L}_{train}(w, \alpha) \tag{4}$$

- Inner level: w is trained to optimality on the training loss for the current α .
- Outer level: α minimizes the held-out validation loss.
- Intuition:
 - w learns to fit the training data,
 - while α selects an architecture that generalizes well.

Since α behaves like a hyperparameter, it is optimized on validation data to avoid overfitting.

Joint optimization: 3.56% error $\rightarrow$ Bilevel DARTS: 2.76%.

7. Approximate architecture gradient

Exact bilevel optimization is infeasible because $w^*(\alpha)$ requires training the network to convergence for every α update. DARTS approximates $w^*(\alpha)$ using one training step:

$$\begin{aligned} & \nabla_{\alpha} \mathcal{L}_{val}(w^*(\alpha), \alpha) \\ & \approx \nabla_{\alpha} \mathcal{L}_{val}(w - \xi \nabla_w \mathcal{L}_{train}(w, \alpha), \alpha) \end{aligned} \tag{6}$$

- Instead of fully solving the inner optimization, DARTS looks one step ahead:
 - $w' = w - \xi \nabla_w \mathcal{L}_{train}(w, \alpha)$
- Then α is updated using the validation loss at this one-step-updated weight w' .
- Second-order DARTS uses this one-step unrolled gradient.
- First-order DARTS sets $\xi = 0$, ignoring the unroll.
 - It is faster but slightly less accurate: 3.00% vs. 2.76% CIFAR-10 error.

8. The DARTS algorithm

Algorithm 1. Differentiable Architecture Search

Construct a mixed operation $\bar{o}^{(i,j)}$ parameterized by $\alpha^{(i,j)}$ for each edge (i, j)

while not converged do

1. Update α by descending $\nabla_{\alpha} L_{\text{val}}(w - \xi \nabla_w L_{\text{train}}(w, \alpha), \alpha)$

(set $\xi = 0$ for the first-order approximation)

2. Update w by descending $\nabla_w L_{\text{train}}(w, \alpha)$

end while

Derive the final architecture: keep the strongest non-zero operation per edge.

Convergence guarantees are not established, but the procedure reaches a fixed point in practice with a suitable choice of ξ .

9. Deriving the discrete architecture

- For each intermediate node, retain the k incoming edges with the largest softmax weight (excluding the zero operation).
- $k = 2$ for convolutional cells, following Zoph et al. (2018); Liu et al. (2018a); Real et al. (2018).
- $k = 1$ for recurrent cells, following Pham et al. (2018b).
- The zero operation is excluded because under batch normalization (Ioffe & Szegedy, 2015), increasing its logit only rescales node outputs and does not affect classification — its softmax weight is therefore underdetermined.

Limitation acknowledged by the authors

There is a discrepancy between the relaxed (soft) architecture used during search and the discrete (hard) architecture used at evaluation. The authors propose annealing the softmax temperature as a possible remedy.

10. Experimental setup

Two-stage protocol

- Search stage. Train an 8-cell network with DARTS for 50 epochs on half the training set, using the other half as validation. Repeat with 4 random seeds; select the cell with the best validation performance.
- Evaluation stage. Stack the chosen cell into a 20-cell network; train from scratch (search-time weights are discarded); report test error.

Datasets

- CIFAR-10 — convolutional cell search.
- ImageNet (mobile setting) — transfer of the CIFAR-10 cell.
- Penn Treebank — recurrent cell search.
- WikiText-2 — transfer of the PTB cell.

11. Discovered cells

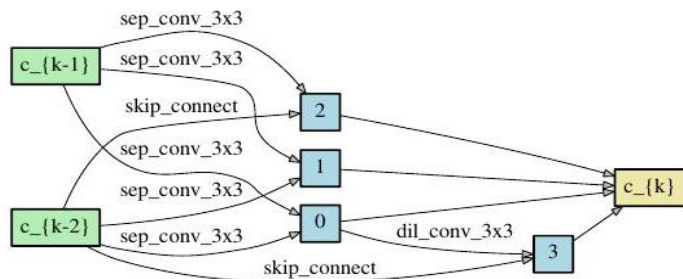


Figure 4. Normal cell, CIFAR-10.

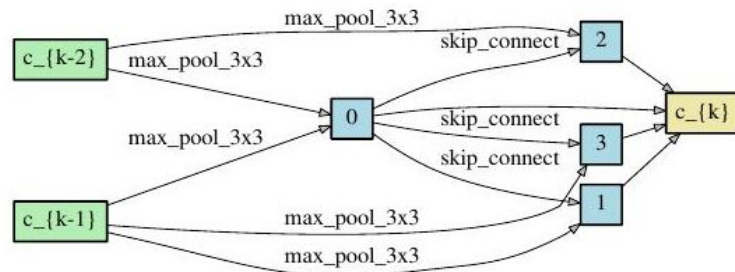


Figure 5. Reduction cell, CIFAR-10.

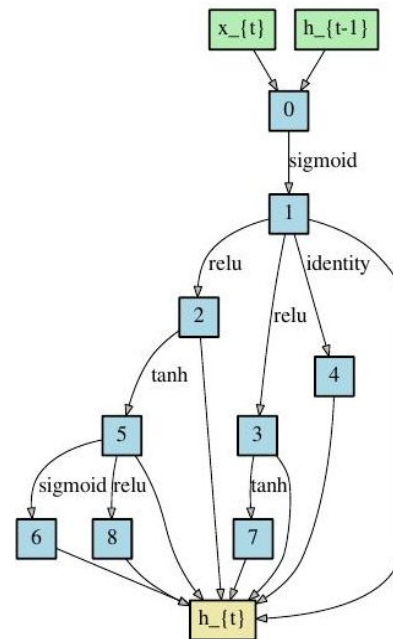


Figure 6. Recurrent cell, PTB.

12. Results — CIFAR-10

<i>Architecture</i>	<i>Test error (%)</i>	<i>Params (M)</i>	<i>Search cost (GPU-d)</i>	<i>Method</i>
DenseNet-BC (Huang et al., 2017)	3.46	25.6	—	manual
NASNet-A (Zoph et al., 2018)	2.65	3.3	2,000	RL
AmoebaNet-B (Real et al., 2018)	2.55	2.8	3,150	evolution
PNAS (Liu et al., 2018a)	3.41	3.2	225	SMBO
ENAS (Pham et al., 2018b)	2.89	4.6	0.5	RL
Random search baseline	3.29	3.2	4	random
<i>DARTS (first-order)</i>	<i>3.00</i>	<i>3.3</i>	<i>1.5</i>	<i>gradient</i>
<i>DARTS (second-order)</i>	<i>2.76</i>	<i>3.3</i>	<i>4</i>	<i>gradient</i>

Table 1. Comparison on CIFAR-10. All entries use cutout augmentation. Adapted from Liu et al. (2019), Table 1.

DARTS attains accuracy comparable with NASNet and AmoebaNet at **roughly three orders of magnitude lower search cost.**

13. Results — ImageNet (mobile setting)

<i>Architecture</i>	<i>Top-1 err (%)</i>	<i>Top-5 err (%)</i>	<i>Params (M)</i>	<i>Search cost (GPU-d)</i>
MobileNet (Howard et al., 2017)	29.4	10.5	4.2	—
NASNet-A (Zoph et al., 2018)	26.0	8.4	5.3	2,000
AmoebaNet-A (Real et al., 2018)	25.5	8.0	5.1	3,150
PNAS (Liu et al., 2018a)	25.8	8.1	5.1	≈225
<i>DARTS (cell searched on CIFAR-10)</i>	26.7	8.7	4.7	4

Table 3. Comparison on ImageNet (mobile setting, $\leq 600\text{M}$ multiply-adds). Adapted from Liu et al. (2019), Table 3.

The cell discovered on **CIFAR-10 transfers to ImageNet** with top-1 error within 1.2 points of AmoebaNet, **at roughly 800× lower search cost** — evidence that the discovered topology is not specific to the source dataset.

14. Results — language modeling (PTB, WikiText-2)

Penn Treebank

<i>Architecture</i>	<i>Test ppl</i>	<i>Params</i>	<i>Cost</i>
LSTM (Melis et al., 2018)	58.3	24M	—
LSTM + 15 MoS (Yang et al., 2018)	56.0	22M	—
NAS (Zoph & Le, 2017)	64.0	25M	1e4 CPU-d
ENAS (Pham et al., 2018b)	58.6	24M	0.5
Random search baseline	59.4	23M	2
<i>DARTS (second-order)</i>	<i>55.7</i>	<i>23M</i>	<i>1</i>

WikiText-2 (transfer)

<i>Architecture</i>	<i>Test ppl</i>	<i>Params</i>	<i>Cost</i>
LSTM (Merity et al., 2018)	66.0	33M	—
LSTM + skip (Melis et al., 2018)	65.9	24M	—
LSTM + 15 MoS (Yang et al., 2018)	63.3	33M	—
ENAS (Pham et al., 2018b)	70.4	33M	0.5
<i>DARTS (cell from PTB)</i>	<i>69.6</i>	<i>33M</i>	<i>1</i>

On PTB, DARTS attains 55.7 perplexity, surpassing prior NAS results (Zoph & Le, 2017; Pham et al., 2018b) and matching the mixture-of-softmaxes language model. Transfer to WikiText-2 is weaker but still improves on ENAS — the authors attribute this to the small size of the source dataset.

15. Discussion

- **Necessity of the bilevel formulation.** Replacing the bilevel objective with joint optimization of α and w over the union of training and validation data raises CIFAR-10 test error from 2.76% to 3.56%, supporting the interpretation of α as a hyperparameter.
- **Strength of random search.** A random-search baseline obtains 3.29% on CIFAR-10 and 59.4 perplexity on PTB, indicating that a substantial part of the result is attributable to the search space itself; DARTS still improves on this baseline meaningfully.
- **Transferability.** Cells searched on CIFAR-10 generalize to ImageNet, and cells searched on PTB to WikiText-2 (more weakly), suggesting that search can be performed on a cheaper proxy task.
- **Acknowledged limitation.** The discrepancy between the continuous architecture searched and the discrete one evaluated is not eliminated; the authors suggest annealing the softmax temperature as future work.

16. Conclusion

- DARTS reformulates neural architecture search as a continuous, differentiable optimization problem solved by gradient descent.
- The same algorithm applies to both convolutional and recurrent architectures.
- On CIFAR-10, ImageNet, Penn Treebank, and WikiText-2 the discovered cells are competitive with the best results from reinforcement learning and evolution, while reducing search cost from thousands of GPU-days to a few.
- Open questions noted by the authors: closing the soft-to-hard discretization gap (e.g. via softmax annealing) and developing performance-aware architecture-derivation schemes that exploit the shared weights learned during search.

References (selected)

- Anandalingam, G. and Friesz, T. L. (1992). Hierarchical optimization. *Annals of Operations Research*, 34(1):1–11.
- Bender, G., Kindermans, P.-J., Zoph, B., Vasudevan, V., and Le, Q. V. (2018). Understanding and simplifying one-shot architecture search. *ICML*.
- Brock, A., Lim, T., Ritchie, J. M., and Weston, N. (2018). SMASH: One-shot model architecture search through hypernetworks. *ICLR*.
- Finn, C., Abbeel, P., and Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. *ICML*.
- Liu, C. et al. (2018a). Progressive neural architecture search. *ECCV*.
- Liu, H., Simonyan, K., and Yang, Y. (2019). DARTS: Differentiable architecture search. *ICLR*.
- Pham, H., Guan, M. Y., Zoph, B., Le, Q. V., and Dean, J. (2018b). Efficient neural architecture search via parameter sharing. *ICML*.
- Real, E., Aggarwal, A., Huang, Y., and Le, Q. V. (2018). Regularized evolution for image classifier architecture search. *arXiv:1802.01548*.
- Zoph, B. and Le, Q. V. (2017). Neural architecture search with reinforcement learning. *ICLR*.
- Zoph, B., Vasudevan, V., Shlens, J., and Le, Q. V. (2018). Learning transferable architectures for scalable image recognition. *CVPR*.